AI Assistant Coding

Assignment 11.2

Name : A.Niteesh **HT. No :** 2303A52341 **Batch:** 32

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

class Stack:

pass

Expected Output:

- A functional stack implementation with all required methods and docstrings.

Code:

```
'''
Genarte a stack class with push pop peek is_empty methods  with docsrings
'''
class Stack:
    """
    A class representing a stack data structure.

    Methods:
    - push(item): Adds an item to the top of the stack.
    - pop(): Removes and returns the item at the top of the stack. Raises an
exception if the stack is empty.
    - peek(): Returns the item at the top of the stack without removing it.
Raises an exception if the stack is empty.
    - is_empty(): Returns True if the stack is empty, False otherwise.
    """

    def __init__(self):
        """Initialize an empty stack."""
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.items.append(item)
```

```
    def pop(self):
        """Remove and return the item at the top of the stack. Raises an
exception if the stack is empty."""
        if self.is_empty():
            raise IndexError("Pop from an empty stack")
        return self.items.pop()

    def peek(self):
        """Return the item at the top of the stack without removing it. Raises
an exception if the stack is empty."""
        if self.is_empty():
            raise IndexError("Peek from an empty stack")
        return self.items[-1]

    def is_empty(self):
        """Return True if the stack is empty, False otherwise."""
        return len(self.items) == 0
# Example usage:
if __name__== "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    print(stack.peek())  # Output: 2
    print(stack.pop())   # Output: 2
    print(stack.is_empty())  # Output: False
    print(stack.pop())   # Output: 1
    print(stack.is_empty())  # Output: True
```

Output:

```
2
2
False
1
True
```

---

ask Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

pass

Expected Output:

• FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Code:

```
'''
Generate a queue class with enqueue dequeue front is_empty methods using
python lists with docstrings
'''
class Queue:
    """
    A class representing a queue data structure.

    Methods:
    - enqueue(item): Adds an item to the end of the queue.
    - dequeue(): Removes and returns the item at the front of the queue.
Raises an exception if the queue is empty.
    - front(): Returns the item at the front of the queue without removing it.
Raises an exception if the queue is empty.
    - is_empty(): Returns True if the queue is empty, False otherwise.
    """

    def __init__(self):
        """Initialize an empty queue."""
        self.items = []

    def enqueue(self, item):
        """Add an item to the end of the queue."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return the item at the front of the queue. Raises an
exception if the queue is empty."""
        if self.is_empty():
            raise IndexError("Dequeue from an empty queue")
        return self.items.pop(0)

    def front(self):
        """Return the item at the front of the queue without removing it.
Raises an exception if the queue is empty."""
        if self.is_empty():
            raise IndexError("Front from an empty queue")
        return self.items[0]

    def is_empty(self):
        """Return True if the queue is empty, False otherwise."""
```

```
        return len(self.items) == 0
# Example usage:
if __name__== "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    print(queue.front())  # Output: 1
    print(queue.dequeue())  # Output: 1
    print(queue.is_empty())  # Output: False
    print(queue.dequeue())  # Output: 2
    print(queue.is_empty())  # Output: True
```

Output:

```
1
1
False
2
True
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

class LinkedList:

pass

Expected Output:

• A working linked list implementation with clear method documentation.

Code:

```
'''
Generate a singly linked list class with insert and display methods with
docstrings
'''
```

```python
class Node:
    """
    A class representing a node in a singly linked list.

    Attributes:
    - data: The value stored in the node.
    - next: A reference to the next node in the list.
    """

    def __init__(self, data):
        """Initialize a node with the given data and set next to None."""
        self.data = data
        self.next = None
class SinglyLinkedList:
    """
    A class representing a singly linked list.

    Methods:
    - insert(data): Inserts a new node with the given data at the end of the
list.
    - display(): Prints the elements of the list in order.
    """

    def __init__(self):
        """Initialize an empty singly linked list."""
        self.head = None

    def insert(self, data):
        """Insert a new node with the given data at the end of the list."""
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def display(self):
        """Print the elements of the list in order."""
        current_node = self.head
        while current_node:
            print(current_node.data, end=' ')
            current_node = current_node.next
        print()  # for a new line after displaying the list
# Example usage:
if __name__== "__main__":
    linked_list = SinglyLinkedList()
```

```
    linked_list.insert(1)
    linked_list.insert(2)
    linked_list.insert(3)
    linked_list.display()  # Output: 1 2 3
```

Output:

```
1 2 3
```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

class BST:

pass

Expected Output:

- BST implementation with recursive insert and traversal methods.

Code:

```
'''
Generate a binary search tree with insert and inorder traversal methods with
docstrings
'''
class TreeNode:
    """
    A class representing a node in a binary search tree.

    Attributes:
    - data: The value stored in the node.
    - left: A reference to the left child node.
    - right: A reference to the right child node.
    """

    def __init__(self, data):
        """Initialize a tree node with the given data and set left and right
to None."""
        self.data  =  data
        self.left  =  None
        self.right = None
class BinarySearchTree:
    """
```

```python
    A class representing a binary search tree.

    Methods:
    - insert(data): Inserts a new node with the given data into the tree while maintaining the binary search tree property.
    - inorder_traversal(): Returns a list of the elements in the tree in sorted order.
    """

    def __init__(self):
        """Initialize an empty binary search tree."""
        self.root = None

    def insert(self, data):
        """Insert a new node with the given data into the tree while maintaining the binary search tree property."""
        if self.root is None:
            self.root = TreeNode(data)
        else:
            self._insert_recursive(self.root, data)

    def _insert_recursive(self, node, data):
        """Helper method to insert a new node recursively."""
        if data < node.data:
            if node.left is None:
                node.left = TreeNode(data)
            else:
                self._insert_recursive(node.left, data)
        else:
            if node.right is None:
                node.right = TreeNode(data)
            else:
                self._insert_recursive(node.right, data)

    def inorder_traversal(self):
        """Return a list of the elements in the tree in sorted order."""
        return self._inorder_recursive(self.root)

    def _inorder_recursive(self, node):
        """Helper method to perform inorder traversal recursively."""
        elements = []
        if node:
            elements.extend(self._inorder_recursive(node.left))
            elements.append(node.data)
            elements.extend(self._inorder_recursive(node.right))
        return elements
# Example usage:
if __name___== "__main__":
```

```
    bst = BinarySearchTree()
    bst.insert(5)
    bst.insert(3)
    bst.insert(7)
    bst.insert(2)
    bst.insert(4)
    bst.insert(6)
    bst.insert(8)
    print(bst.inorder_traversal())  # Output: [2, 3, 4, 5, 6, 7, 8]
```

Output:

```
[2, 3, 4, 5, 6, 7, 8]
```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable:

pass

Expected Output:

• Collision handling using chaining, with well-commented methods.

Code:

```
'''
Genarate hash table with insert , search, and delete methods with docstrings
'''
class HashTable:
    """
    A class representing a hash table data structure.

    Methods:
    - insert(key, value): Inserts a key-value pair into the hash table.
    - search(key): Returns the value associated with the given key. Raises an
exception if the key is not found.
    - delete(key): Removes the key-value pair associated with the given key
from the hash table. Raises an exception if the key is not found.
    """
```

```python
    def __init__(self, size=10):
        """Initialize a hash table with a specified size."""
        self.size = size
        self.table = [None] * self.size

    def _hash(self, key):
        """Generate a hash for the given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert a key-value pair into the hash table."""
        index = self._hash(key)
        if self.table[index] is None:
            self.table[index] = [(key, value)]
        else:
            for i, (k, v) in enumerate(self.table[index]):
                if k == key:
                    self.table[index][i] = (key, value)  # Update existing key
                    return
            self.table[index].append((key, value))  # Add new key-value pair

    def search(self, key):
        """Return the value associated with the given key. Raises an exception
if the key is not found."""
        index = self._hash(key)
        if self.table[index] is not None:
            for k, v in self.table[index]:
                if k == key:
                    return v
        raise KeyError("Key not found")

    def delete(self, key):
        """Remove the key-value pair associated with the given key from the
hash table. Raises an exception if the key is not found."""
        index = self._hash(key)
        if self.table[index] is not None:
            for i, (k, v) in enumerate(self.table[index]):
                if k == key:
                    del self.table[index][i]
                    return
        raise KeyError("Key not found")

# Example usage:
if __name__== "__main__":
    hash_table = HashTable()
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    print(hash_table.search("name"))  # Output: Alice
```

```
    print(hash_table.search("age"))    # Output: 30
    hash_table.delete("name")
    try:
        print(hash_table.search("name")) # This will raise an exception
    except KeyError as e:
        print(e)  # Output: Key not found
```

Output:

```
Alice
30
'Key not found'
```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

pass

Expected Output:

• Graph with methods to add vertices, add edges, and display connections.

Code:

```
'''
implement graph using adjacency list
'''
class Graph:
    def __init__(self):
        self.graph = {}

    def add_edge(self, u, v):
        if u not in self.graph:
            self.graph[u] = []
        if v not in self.graph:
            self.graph[v] = []
        self.graph[u].append(v)
        self.graph[v].append(u)

    def display(self):
        for vertex in self.graph:
            print(f"{vertex}: {self.graph[vertex]}")
# Example usage:
```

```
if __name__== "__main__":
    g = Graph()
    g.add_edge(0, 1)
    g.add_edge(0, 2)
    g.add_edge(1, 2)
    g.add_edge(1, 3)
    g.display()
```

Output:

```
0: [1, 2]
1: [0, 2, 3]
2: [0, 1]
3: [1]
```

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

class PriorityQueue:

pass

Expected Output:

• Implementation with enqueue (priority), dequeue (highest priority),

and display methods.

Code:

```
'''
implement priority queue using heapq modeule with enqueue and dequeue
operations and display methods
'''
import heapq
class PriorityQueue:
    def __init__(self):
        self.elements = []

    def enqueue(self, item, priority):
        heapq.heappush(self.elements, (priority, item))

    def dequeue(self):
        if not self.is_empty():
```

```
            return heapq.heappop(self.elements)[1]
        raise IndexError("Priority Queue is empty")
    def is_empty(self):
        return len(self.elements) == 0
    def display(self):
        print("Priority Queue:")
        for priority, item in self.elements:
            print(f"Item: {item}, Priority: {priority}")
# Example usage:
if __name__== "__main__":
    pq = PriorityQueue()



    pq.enqueue("Task 1", 2)
    pq.enqueue("Task 2", 1)
    pq.enqueue("Task 3", 3)
    pq.display()
    print("Dequeue:", pq.dequeue())  # Output: Task 2
    pq.display()
```

Output:

```
Priority Queue:
Item: Task 2, Priority: 1
Item: Task 1, Priority: 2
Item: Task 3, Priority: 3
Dequeue: Task 2
Priority Queue:
Item: Task 1, Priority: 2
Item: Task 3, Priority: 3
```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Sample Input Code:

class DequeDS:

pass

Expected Output:

• Insert and remove from both ends with docstrings.

Code:

```
'''
```

```
implement a double-ended queue using collections.deque and insert and remove
from both ends with docstrings.
'''
from collections import deque
class DequeDs:
    """
    A class representing a double-ended queue (deque) data structure.

    Methods:
    - insert_front(item): Inserts an item at the front of the deque.
    - insert_rear(item): Inserts an item at the rear of the deque.
    - remove_front(): Removes and returns the item at the front of the deque.
Raises an exception if the deque is empty.
    - remove_rear(): Removes and returns the item at the rear of the deque.
Raises an exception if the deque is empty.
    - display(): Displays the contents of the deque.
    """

    def __init__(self):
        """Initialize an empty deque."""
        self.deque = deque()

    def insert_front(self, item):
        """Insert an item at the front of the deque."""
        self.deque.appendleft(item)

    def insert_rear(self, item):
        """Insert an item at the rear of the deque."""
        self.deque.append(item)

    def remove_front(self):
        """Remove and return the item at the front of the deque. Raises an
exception if the deque is empty."""
        if not self.is_empty():
            return self.deque.popleft()
        raise IndexError("Deque is empty")

    def remove_rear(self):
        """Remove and return the item at the rear of the deque. Raises an
exception if the deque is empty."""
        if not self.is_empty():
            return self.deque.pop()
        raise IndexError("Deque is empty")

    def is_empty(self):
        """Check if the deque is empty."""
        return len(self.deque) == 0
```

```
    def display(self):
        """Display the contents of the deque."""
        print("Deque contents:", list(self.deque))
# Example usage:
if __name___== "__main__":
    deque_ds = DequeDs()
    deque_ds.insert_rear("Task 1")
    deque_ds.insert_rear("Task 2")
    deque_ds.insert_front("Task 0")
    deque_ds.display()  # Output: Deque contents: ['Task 0', 'Task 1', 'Task
2']
    print("Remove front:", deque_ds.remove_front()) # Output: Task 0
    print("Remove rear:", deque_ds.remove_rear())    # Output: Task 2
    deque_ds.display()  # Output: Deque contents: ['Task 1']
    print("Is deque empty?", deque_ds.is_empty()) # Output: False
    deque_ds.remove_front() # Remove remaining item
    print("Is deque empty?", deque_ds.is_empty()) # Output: True
```

Output:

```
Deque contents: ['Task 0', 'Task 1', 'Task 2']
Remove front: Task 0
Remove rear: Task 2
Deque contents: ['Task 1']
Is deque empty? False
Is deque empty? True
```

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.

2. Event Registration System – Manage participants in events with quick search and removal.

3. Library Book Borrowing – Keep track of available books and their due dates.

4. Bus Scheduling System – Maintain bus routes and stop connections.

5. Cafeteria Order Queue – Serve students in the order they arrive.

Code:

```
# generate a python code for bus scheduling system using graph data structure
class Graph:
    def __init__(self):
        self.graph = {}
    def add_edge(self, u, v):
        if u not in self.graph:
            self.graph[u] = []
        if v not in self.graph:
            self.graph[v] = []
        self.graph[u].append(v)
        self.graph[v].append(u)  # For undirected graph
    def bfs(self, start):
        visited = set()
        queue = [start]
        while queue:
            vertex = queue.pop(0)
            if vertex not in visited:
                print(vertex, end=' ')
                visited.add(vertex)
                queue.extend(set(self.graph[vertex]) - visited)
    def dfs(self, start, visited=None):
        if visited is None:
            visited = set()
        if start not in visited:
            print(start, end=' ')
            visited.add(start)
            for neighbor in self.graph[start]:
                self.dfs(neighbor, visited)
# example
bus_schedule = Graph()
bus_schedule.add_edge("Stop A", "Stop B")
bus_schedule.add_edge("Stop A", "Stop C")
bus_schedule.add_edge("Stop B", "Stop D")
bus_schedule.add_edge("Stop C", "Stop D")
print("Bus route from Stop A:")
bus_schedule.bfs("Stop A")  # Output: Stop A Stop B Stop C Stop D
print("\nBus route from Stop A (DFS):")
bus_schedule.dfs("Stop A")  # Output: Stop A Stop B Stop D Stop C
```

Task Description #10: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

CODE:

```python
#generate a python code for hash table implementation for medical records storage
class HashTable:
    def __init__(self, size=100):
        self.size = size
        self.table = [None] * self.size

    def _hash(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self._hash(key)
        if self.table[index] is None:
            self.table[index] = [(key, value)]
        else:
            for i, (k, v) in enumerate(self.table[index]):
                if k == key:
                    self.table[index][i] = (key, value)  # Update existing key
                    return
            self.table[index].append((key, value))  # Add new key-value pair

    def get(self, key):
        index = self._hash(key)
        if self.table[index] is not None:
            for k, v in self.table[index]:
                if k == key:
                    return v
        return None  # Key not found

    def delete(self, key):
        index = self._hash(key)
        if self.table[index] is not None:
            for i, (k, v) in enumerate(self.table[index]):
                if k == key:
                    del self.table[index][i]  # Remove the key-value pair
                    return True
        return False  # Key not found
```

```python
        return False  # Key not found
# Example usage
if __name__ == "__main__":
    medical_records = HashTable()
    medical_records.insert("patient_001", {"name": "John Doe", "age": 30, "condition": "Flu"})
    medical_records.insert("patient_002", {"name": "Jane Smith", "age": 25, "condition": "Cold"})

    print(medical_records.get("patient_001"))  # Output: {'name': 'John Doe', 'age': 30, 'condition': 'Flu'}
    print(medical_records.get("patient_002"))  # Output: {'name': 'Jane Smith', 'age': 25, 'condition': 'Cold'}

    medical_records.delete("patient_001")
    print(medical_records.get("patient_001"))  # Output: None
```

Output:

```
{'name': 'John Doe', 'age': 30, 'condition': 'Flu'}
{'name': 'Jane Smith', 'age': 25, 'condition': 'Cold'}
None
```

Task Description #11: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.

2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.

3. Vehicle Registration Lookup – Instant access to vehicle details.

4. Road Network Mapping – Roads and intersections connected logically.

5. Parking Slot Availability – Track available and occupied slots.

Code:

```python
#generate a python code for priority queue implementation for vehicle priority handling
class PriorityQueue:
    def __init__(self):
        self.elements = []

    def is_empty(self):
        return not self.elements

    def put(self, item, priority):
        self.elements.append((priority, item))
        self.elements.sort(key=lambda x: x[0])  # Sort by priority

    def get(self):
        if not self.is_empty():
            return self.elements.pop(0)[1]  # Return the item with the highest priority
        raise IndexError("Priority Queue is empty")
# Example usage
if __name__ == "__main__":
    pq = PriorityQueue()
    pq.put("Vehicle A", 2)  # Lower number means higher priority
    pq.put("Vehicle B", 1)
    pq.put("Vehicle C", 3)

    print(pq.get())  # Output: Vehicle B (highest priority)
    print(pq.get())  # Output: Vehicle A
    print(pq.get())  # Output: Vehicle C
    # print(pq.get())  # This will raise an error since the queue is empty
```

Output:

```
Vehicle B
Vehicle A
Vehicle C
```

Task 12: An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

```python
#generate a python code for graph implementation for delivery route planning
class Graph:
    def __init__(self):
        self.graph = {}

    def add_edge(self, u, v, weight=1):
        if u not in self.graph:
            self.graph[u] = []
        if v not in self.graph:
            self.graph[v] = []
        self.graph[u].append((v, weight))
        self.graph[v].append((u, weight))  # For undirected graph

    def dijkstra(self, start):
        import heapq
        queue = [(0, start)]
        distances = {node: float('inf') for node in self.graph}
        distances[start] = 0
        while queue:
            current_distance, current_node = heapq.heappop(queue)
            if current_distance > distances[current_node]:
                continue
            for neighbor, weight in self.graph[current_node]:
                distance = current_distance + weight
                if distance < distances[neighbor]:
                    distances[neighbor] = distance
                    heapq.heappush(queue, (distance, neighbor))
        return distances
# Example usage
if __name__ == "__main__":
    g = Graph()
    g.add_edge("Warehouse", "Customer A", 5)
    g.add_edge("Warehouse", "Customer B", 10)
    g.add_edge("Customer A", "Customer B", 2)

    distances = g.dijkstra("Warehouse")
    print(distances)  # Output: {'Warehouse': 0, 'Customer A': 5, 'Customer B': 7}
```

OUTPUT:

```
{'Warehouse': 0, 'Customer A': 5, 'Customer B': 7}
```

Task Description #14: Smart Banking Management System – Data Structure Challenge

A bank plans to develop a Smart Banking System that includes:

1. Customer Service Token System – Customers are served in order of arrival.
2. Loan Approval Priority Handling – High-value or urgent loans processed first.
3. Account Information Lookup – Fast retrieval of customer details using Account Number.
4. Transaction History Management – Maintain chronological transaction records.
5. ATM Network Connectivity – Represent connections between

ATMs across cities.
Student Task:
• For each feature, select the most appropriate data structure from the list below:
o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque
• Justify your choice in 2–3 sentences per feature.

Feature	**Data Structure**	**Justification**
Customer Token System	Queue	Customers served in order of arrival.
Loan Approval Priority	Priority Queue	High-value loans processed first.
Account Lookup	Hash Table	Fast retrieval using account number.
Transaction History	Linked List	Transactions stored sequentially.
ATM Connectivity	Graph	ATMs across cities connected via network.

• Implement one selected feature as a working Python program with AI-assisted code generation.
Expected Output:
• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

```python
#generate a python code for queue  implementation for Customer Service Token System
class Queue:
    def __init__(self):
        self.elements = []

    def is_empty(self):
        return not self.elements

    def enqueue(self, item):
        self.elements.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.elements.pop(0)  # Remove and return the first item
        raise IndexError("Queue is empty")
# Example usage
if __name__ == "__main__":
    customer_queue = Queue()
    customer_queue.enqueue("Customer 1")
    customer_queue.enqueue("Customer 2")
    customer_queue.enqueue("Customer 3")

    print(customer_queue.dequeue())  # Output: Customer 1
    print(customer_queue.dequeue())  # Output: Customer 2
    print(customer_queue.dequeue())  # Output: Customer 3
    # print(customer_queue.dequeue())  # This will raise an error since the queue is empty
```

```
Customer 1
Customer 2
Customer 3
```

Task Description #15: Online Learning Management System – Data Structure Selection

An online education platform wants to develop a Learning Management System that handles:

1. Student Login Authentication – Quick validation of user credentials.
2. Assignment Submission Queue – Submissions processed in order of upload.
3. Top Performer Ranking – Rank students based on scores.
4. Course Prerequisite Structure – Represent subject dependencies.
5. Undo Feature in Online Editor – Reverse recent actions.

Student Task:

• Select the most appropriate data structure for each feature from the given list.

• Justify your selection in 2–3 sentences per feature.

Feature	**Data Structure**	**Justification**
Student Login Authentication	Hash Table	Quick username/password lookup.
Assignment Submission	Queue	Submissions processed in order received.
Top Performer Ranking	Priority Queue	Students ranked by highest score.

Feature	Data Structure	Justification
Course Prerequisites	Graph	Courses depend on other courses.
Undo Feature	Stack	Undo follows LIFO order of actions.

• Implement one selected feature in Python using AI-assisted code generation.
Expected Output:
• Feature → Data Structure → Justification table.
• Functional Python program with proper documentation.

```python
#generate a python code for undo feature in online text editor using stack data structure
class Stack:
    def __init__(self):
        self.elements = []

    def is_empty(self):
        return not self.elements

    def push(self, item):
        self.elements.append(item)

    def pop(self):
        if not self.is_empty():
            return self.elements.pop()  # Remove and return the last item
        raise IndexError("Stack is empty")
# Example usage
if __name__ == "__main__":
    undo_stack = Stack()
    undo_stack.push("Type 'Hello'")
    undo_stack.push("Type 'World'")
    undo_stack.push("Delete 'World'")

    print(undo_stack.pop())  # Output: Delete 'World'
    print(undo_stack.pop())  # Output: Type 'World'
    print(undo_stack.pop())  # Output: Type 'Hello'
    # print(undo_stack.pop())  # This will raise an error since the stack is empty
```

```
Delete 'World'
Type 'World'
Type 'Hello'
```